instituto
superior de
engenharia
de lisboa

Modular Platform for Data Acquisition and Analysis Based on CAN for Distributed Vehicular Systems

Luís Henrique Ferreira
Gonçalo José Ferraz

Project Work in the Field of Computer and Informatics Engineering

Supervisors:
Eng. David Emanuel Ribeiro Velez

Jury:
President: Doutor Fernando Sousa
Vowels: Doutor Rui Duarte
Eng. David Emanuel Ribeiro Velez

Julho of 2026

Modular Platform for Data Acquisition and Analysis Based on CAN for Distributed Vehicular Systems

Luis Henrique Ferreira
Gonçalo José Ferraz

Project in the Field of Computer and Informatics Engineering

Supervisors:
Eng. David Emanuel Ribeiro Velez

Jury:
President: Doutor Fernando Sousa
Vowels: Doutor Rui Duarte
Eng. David Emanuel Ribeiro Velez

Julho of 2026

AI usage disclosure

In this project we utilized AI modules in order to research and assist with the development, more specifically in the context of:

Bluetooth Integration with BlueZ Framework Research
Android UI Presentation
CAN Decoding

Statement of Integrity

I declare that this project work is the result of my personal and independent research. Its content is original, and all sources listed in the bibliographic references were consulted and are duly mentioned in the text. I further declare that all scientific and technical references relevant to the development of the work are duly cited and included in the bibliographic references.

The authors

Luís Henrique Borges Coury Cavaleiro de Ferreira
Gonçalo José Soeiro Ferraz

Contents

List of Tables

List of Figures

List of Acronyms

ADV Ignition Advance

AFR Air-Fuel Ratio

API Application Programming Interface

BCU Battery Unit Control

BLE Bluetooth Low-Energy

CAN Controller Area Network

CLT Coolant Temperature

CPU Central Processing Unit

CRC Cyclic Redundancy Check

DB Database

ECU Electronic Control Unit

EGO Exhaust Gas Oxygen

EV Eletric Vehicle

FP Fuel Pump

GATT Generic Attribute Profile

GPIO General Purpose Input/Output

GUI Graphical User Interface

IAT Intake Air Temperature

ICE Internal Combustion Engine

JSON JavaScript Object Notation

MAP Manifold Absolute Pressure

PWM Pulse Width Modulation

RAM Random Access Memory

RPM Revolutions Per Minute

RT Real-Time

SQL Structured Query Language

SoC System on Chip

TPS Throttle Position Sensor

UART Universal Asynchronous Receiver-Transmitter

USB Universal Serial Bus

UUID Universally Unique Identifier

VCU Vehicle Control Unit

VSS Vehicle Speed Sensor

VE Volumetric Efficiency

Chapter 1

Introduction

The document describes the development of a modular data acquisition and analysis platform based on Controller Area Network (CAN) communication, designed for application in distributed vehicular systems. Modern vehicles exhibit increasing electronic complexity and a high distribution of control units and sensors, making it essential to provide tools and solutions capable of centralizing, interpreting, and exploring data efficiently. This project proposes an integrated approach that combines real-time data acquisition, structured storage, and analysis and visualization tools, with the objective of improving the understanding of vehicle behavior and supporting technical decision-making processes in demanding environments such as motorsport.

1.1 Context and Framework

This project is developed within the scope of the Project and Seminar curricular unit of the Bachelor's Degree in Computer and Informatics Engineering (LEIC) at Instituto Superior de Engenharia de Lisboa, during the 2025/2026 academic year. The project's application domain focuses on distributed vehicular systems, with particular emphasis on the Motorsport and aftermarket sectors. These contexts are characterized by demanding requirements in terms of real-time data acquisition, transmission, and analysis, which are essential for performance monitoring, fault diagnosis, and system optimization. The increasing electronic complexity and the vast diversity of systems used in modern vehicles and/or motorsport/enthusiast applications make the development of modular, flexible, and scalable platforms essential.

The platform is developed based on the Raspberry Pi ecosystem, taking advantage of its versatility, low cost, and extensive support documentation. The project primarily uses the *Python* and *Bash* programming languages, enabling a hybrid approach that combines ease of development with execution capability in embedded systems. This technological choice aims to facilitate rapid prototyping while ensuring flexibility for adaptation to different usage scenarios.

A distinguishing element of this project is its validation environment, which takes place in the real-world context of a vehicle equipped with a *RusEFI* ECU, as well as within a Formula Student project. This framework allows not only the testing of the solution under conditions close to real operation, but also dealing with practical challenges such as hardware constraints, electromagnetic noise, communication limitations, and robustness and reliability requirements. Integration into a competition vehicle also adds a critical real-time and performance dimension, where data-driven decisions can have a direct impact on achieved results.

In summary, this project is positioned at the intersection of software engineering and embedded systems applied to the automotive domain, seeking to address the growing

need for modular and efficient solutions for data acquisition and analysis in distributed vehicular systems.

1.2 Problem and Motivation

The development of modern vehicular systems raises a set of significant challenges regarding data acquisition, integration, and interpretation. One of the main identified issues is the strong dependence on the vehicle's Electronic Control Units (ECUs), which often operate as closed systems, limiting direct and flexible access to information. In this context, the need arises to create an autonomous and fully parameterizable solution capable of ensuring independence from these units, with the objective of enabling the integration of different applications through database access via Bluetooth. The use of CAN databases (DBC) plays a central role in this approach, allowing the structured decoding and interpretation of messages without requiring direct intervention in the ECUs.

At the same time, the increasing complexity of distributed vehicular systems introduces additional challenges in data management. In a modern vehicle, multiple subsystems coexist, such as the ECU, VCU (Vehicle Control Unit), and BCU (Battery Control Unit), each responsible for different critical functions. These subsystems often communicate through different protocols, such as CAN, serial communication, Wi-Fi, or Bluetooth, resulting in a heterogeneous and fragmented ecosystem. The absence of a unifying platform makes the centralization and synchronization of information difficult, compromising its practical usefulness.

In competitive environments such as Formula Student, this data fragmentation translates into a concrete limitation: the difficulty of correlating information originating from different sources. Without robust integrated analysis tools, it becomes challenging to identify performance patterns, understand the vehicle's dynamic behavior, or detect anomalies in a timely manner. This limitation may have a direct impact on the vehicle optimization process, reducing the effectiveness of the technical decisions made by the team.

Thus, the central motivation of the project lies in the creation of a distributed platform capable of simultaneously transforming large volumes of raw data into structured, clear, actionable, and persistent information. The goal is not only to collect data, but also to provide tools that enable its efficient analysis and interpretation. By facilitating the visualization, correlation, and processing of information, the proposed solution aims to support more informed decision-making, contributing to the continuous improvement of the performance and reliability of vehicular systems.

1.3 Objectives

This project establishes clearly defined objectives, with success assessed through tangible, result-oriented outcomes, culminating in the development and real-world validation of a functional platform:

- **Design a modular and scalable architecture**, that facilitates the integration of the platform to different vehicles, sensors, and technical requirements, minimizing the need for structural changes to the system.
- **Develop a CAN-based data acquisition system**, with the ability to decode messages using DBC files, ensuring reliable real-time reading of relevant vehicle signals.

- **Create an efficient and continuous data logging system**, enabling structured recording of data during extended sessions, without information loss and with guarantees of integrity. Develop a modular real-time dashboard for data visualization, allowing monitoring of critical vehicle parameters with low latency and high clarity, and configurable according to the use context.
- **Implement a multichannel data integration mechanism**, capable of aggregating information from different subsystems and protocols (CAN, UART, and Bluetooth), ensuring consistent time synchronization between sources.
- **Implement data analysis tools**, including statistical analysis methods, session comparison, and anomaly detection, with the aim of extracting relevant information from the collected data.
- **Ensure system robustness and reliability**, guaranteeing stable operation in demanding environments subject to noise, vibration, and power fluctuations. Validate the solution in a real environment through its integration into a vehicle within a Formula Student context, demonstrating its practical applicability and performance under operational conditions.

1.4 Document Structure

This document is structured to demonstrate an understanding of the research conducted, the necessary knowledge for its comprehension, the proposed solution, and the results obtained. More specifically:

- **Chapter 2 - State of the Art:** Reviews the necessary knowledge for the proper interpretation of the thesis, from the inner-workings of a controller area network and how their vehicular information is revealed, the process and analysis of the sensor information to the bluetooth information transfer protocols
- **Chapter 3 - Implementation:** Details of the design choices, experimental setup, system and database architecture and data analysis
- **Chapter 4 - Results and Validation:** Experimental results of the project, detailing the experimental environment and the results given
- **Chapter 5 - Conclusion:** Final thoughts about the project design and implementation and future possible changes to better optimize and develop the project

Chapter 2

State of the Art

Modern vehicles rely on a distributed electronic architecture composed of multiple interconnected control systems that continuously monitor and manage vehicle operation. At the core of this architecture are ECUs, which process data from numerous sensors and coordinate the operation of vehicle subsystems to ensure optimal performance, efficiency, safety, and compliance with regulatory requirements. ECUs are responsible for controlling engine sensors and actuators, as well as providing various vehicle operating parameters such as engine speed (RPM), pressures, temperatures, air-fuel mixture, and other relevant indicators [1].

2.1 Vehicular Communication Networks

2.1.1 Electronic Control Unit

It functions as a small embedded computer, acquiring information from sensors, processing this data in real time, and sending commands to actuators. Depending on its purpose, an ECU may control the engine, transmission, ABS braking system, stability control, power steering, airbags, and many other systems found in modern automobiles [1].

The development of ECUs became increasingly relevant during the 1970s and 1980s, driven by the need to improve engine efficiency and reduce pollutant emissions.

Before the development of ECU, automotive systems were mostly mechanical. As vehicles became more a more complex and had more stict regulations, there was a need for more precise electronic solutions, leading to the creation of the first ECUs

2.1.2 Controller Area Network

The Controller Area Network (CAN) Network is a vehicle message-based communication protocol design to allow an ECU to communicate to other devices without the need for a central computer and to enable a fast and reliable exchange of information between electronic modules [2].

This protocol was developed by Robert Bosch GmbH in the 1980s as a response to the increasing complex vehicular systems of the time. Over time, the CAN protocol became the standard due to it's simplicity and ability to operate in electrically noisy environment.

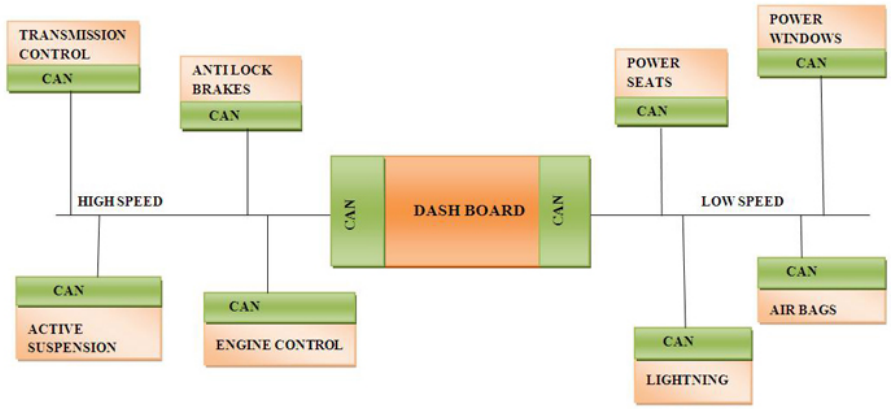


Figure 2.1: CAN Application Example [3]

CAN protocol is a message-based protocol. This means the data of a vehicule is transmitted in messages identified by a unique ID and available for every electronic module to read, instead of being sent to specific devices. Any ECU on the network can broadcast messages, and any other ECU can choose to listen to the messages relevant to it.

These messages, known as frame, follow a specific format as to ensure consistency:

- ID - Unique message identifier
- DLC (Data Length Code) - Specifies the data length in bytes
- Data Field - Contain the information payload
- CRC (Cyclic Redundancy Check) - Used for error and data integrity verification
- ACK Slot - Confirms reception of at least one node
- EOF (End of Frame) - Signals the end of the message

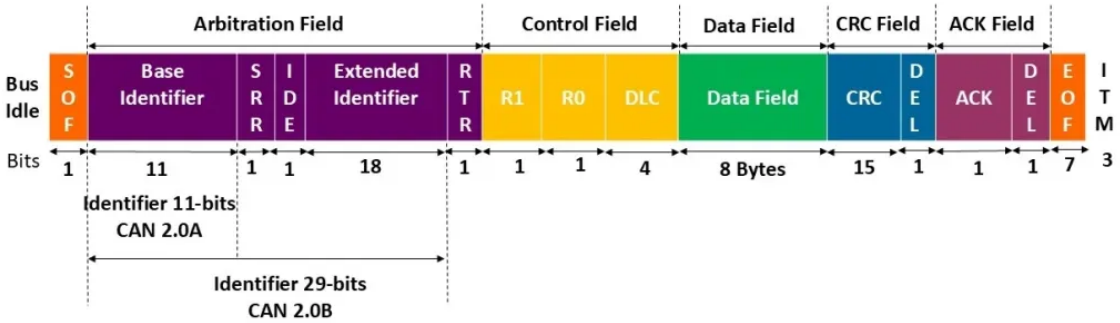


Figure 2.2: CAN Standard Data Frame [4]

2.1.3 Controller Area Network Data Base

CAN databases, commonly represented as DBC files, play a fundamental role in the interpretation and use of CAN bus data in modern automotive and embedded systems. A CAN database defines the structure and semantics of CAN messages, enabling the translation of raw binary frames into meaningful physical signals. A CAN frame, by itself, only contains an identifier and a sequence of data bytes. Without additional context, this

data is not directly interpretable. CAN databases address this limitation by providing a structured description of each message, including its identifier, length, and the layout of individual signals within the payload. Each signal is defined through parameters such as bit position, bit length, scaling factor, offset, and engineering units. This makes it possible to convert raw byte streams into values such as engine speed in revolutions per minute, throttle position, or air fuel ratio [5]. One of the primary use cases of CAN databases is real-time data decoding. In systems such as ECU monitoring platforms or vehicle telemetry applications, incoming CAN frames are matched against database definitions and decoded into structured data. This enables the development of dashboards, logging systems, and diagnostic tools that present meaningful information to the user. In the context of the proposed solution, the DBC file is used to decode messages from a rusefi ECU, translating low level CAN frames into a normalized vehicle state. Another relevant application is data logging and offline analysis. CAN databases allow raw CAN traffic to be recorded while preserving the ability to decode it at a later stage. This is particularly useful for debugging, performance evaluation, and research activities. Since decoding logic is defined externally in the database, previously recorded data can be reinterpreted using updated signal definitions without requiring any modification to the stored dataset. CAN databases are also widely used in simulation and testing environments. By defining message structures and signal properties, DBC files allow synthetic CAN traffic to be generated in a controlled manner. This capability is important in hardware in the loop and software validation scenarios, where realistic communication patterns are required without relying on physical ECUs. Additionally, DBC files contribute to standardization and interoperability across different tools and systems. Multiple applications such as CAN analyzers, embedded controllers, and data acquisition platforms can use the same database to ensure consistent interpretation of CAN messages. This reduces ambiguity and simplifies integration in complex systems involving multiple components and suppliers [6].

2.1.4 Why CAN is the Preferred Method for Intra-Vehicular Communication

Controller Area Network (CAN) is widely adopted in intra-vehicular communication due to its deterministic behavior, fault tolerance, and efficient bus architecture. It is specifically designed for distributed embedded systems where multiple ECUs must exchange time-critical information over a shared medium [7].

CAN employs a multi-master, message-oriented communication model over a differential two-wire bus, which increases noise immunity in electrically harsh automotive environments. Reliability is ensured through multiple built-in mechanisms, including CRC, bit stuffing, frame validation, and automatic retransmission upon error detection. Fault confinement is also supported, allowing malfunctioning nodes to be isolated from the network.

A key feature of CAN is its non-destructive bitwise arbitration mechanism, which guarantees deterministic access to the bus. Message priority is encoded in the identifier field, where lower numerical values correspond to higher priority. During arbitration, nodes transmit simultaneously and monitor the bus, withdrawing transmission if a dominant bit is overwritten by a recessive bit. This ensures that the highest priority message is transmitted without collision or delay.

In terms of efficiency, CAN minimizes wiring complexity by using a shared bus topology, reducing both system cost and physical weight. It supports data rates up to 1 Mbps in Classical CAN, which is sufficient for most control applications. The protocol also allows both 11-bit and 29-bit identifiers, enabling flexible message addressing schemes, as shown in Figure 2.2.

2.1.5 Standalone ECU

Standalone ECUs are aftermarket electronic control systems designed to fully replace the original equipment manufacturer (OEM) ECU. Their primary purpose is to provide greater flexibility, configurability, and control over engine operation, particularly in high-performance and motorsport applications.

Unlike OEM ECUs, which are typically optimized for emissions, reliability, and mass production constraints, standalone ECUs allow direct access to key engine parameters such as fuel injection, ignition timing, boost control, and sensor calibration. This enables precise tuning of engine behaviour according to specific requirements, such as performance optimization, component modifications, or experimental configurations.

The motivation for using standalone ECUs arises from the need for adaptability and transparency. In motorsport applications such as Formula Student, where vehicles are custom-built and continuously refined, engineers require full control over the engine management strategy. Standalone ECUs provide this capability by supporting user-defined maps, real-time parameter adjustment, and integration with external systems through communication protocols such as CAN.

2.1.6 Speeduino/RusEFI ECUs

Speeduino and RusEFI are open-source standalone ECU platforms designed to provide flexible and low-cost engine management solutions. Both systems aim to replicate and extend the functionality of commercial standalone ECUs, while maintaining full transparency and user control over hardware and software [8].

Speeduino is based on Arduino-compatible hardware and focuses on simplicity and accessibility. It provides core engine control features such as fuel injection, ignition timing, and sensor processing, making it a popular choice for educational projects and low-budget motorsport applications. Its architecture allows easy integration with existing tools such as TunerStudio, enabling real-time tuning and data logging.

RusEFI, on the other hand, targets more advanced use cases with a higher level of performance and configurability. It typically runs on more powerful microcontrollers and offers extended features, including native CAN support, higher processing capabilities, and more precise control strategies. This makes it suitable for more demanding applications, such as competitive motorsport or complex engine setups [9].

The main motivation behind both platforms is to provide open, customizable alternatives to proprietary ECUs. They allow users to modify control algorithms, extend communication interfaces, and integrate telemetry systems, such as the one proposed in this work. As a result, they are particularly well suited for environments like Formula Student, where flexibility, experimentation, and cost constraints are critical factors.

2.2 Embedded System's Development Platforms

2.2.1 Raspberry Pi Ecosystem

For the controller of our project, we decide to utilize a Raspberry Pi Zero 2W [10]. This decision was made due to several reasons:

- **Full Linux Computer:** The controller will have to implement several different tasks at once such as:
  - Real-time data aquisition from the ECU
  - Data processing like filtering and parsing signals

  - Database logging and managment
  - UI rendering
  - Real-time bluetooth data transfer

  A Raspberry Pi system provides a good flexibility to develop our application while being able to build our application with the features above.

- **Fast processor and decent memory:** The Raspberry Pi Zero 2 W incorporates a quad-core 64-bit Arm Cortex-A53 CPU clocked at 1GHz, paired with 512MB of LPDDR2 SDRAM. This package provides the Raspberry Pi Zero 2 W with good performance for our application, and having multiple cores will help with our distributed system aproach.
- **Easy communication with hardware:** The Raspberry Pi provides a USB serial port with the possibility to use UART pins already built-in to the computer
- **Networking:** This computer provides a Bluetooth Low Energy (BLE) system which allows the transfer of the CAN signal information easily to a connected mobile device

With all this said, we choose the Raspberry Pi for its fast performance, and affordable price. Another deciding factor in the selection of this computer was the extensive community support, along with the large number of available open-source projects and software resources. Although the selected controller unit is significantly more capable in terms of computing power than the basic project requirements might suggest, this additional processing capability provides greater flexibility and may prove valuable for future project development, feature expansion, and subsequent upgrades.

2.2.2 Utilized Programming Languages

Support for multiple programming languages was a key design consideration, allowing each component to leverage the strengths of the most appropriate language while mitigating its inherent limitations:

- **Python:** Used for the controller application for a ECU data pipeline, controller logic and bluetooth integration due to it's simplicity for serial communication, data parsing and bluetooth features as well as extensive UI libraries;
- **SQLite:** Although Python is used for making requests to the database, said requests are in SQL using a SQLite variant due to being a light-weight alternative to variants like Postgres, for the simplicity of easy for there is no need for a database and instead SQLite stores information in a single file and has extremely fast writes which is perfect for storing the CAN dataframes obtained;
- **Kotlin:** Kotlin was chosen as the primary language for mobile application development due to its seamless integration with the Android platform, support for low-latency applications, comprehensive networking libraries, and the modern Jetpack Compose UI framework.

2.3 Data Analysis Techniques in Motorsport

Data analysis plays an important role in motorsport, where vehicle performance and reliability depend on the ability to interpret large amounts of information collected during

testing and competition. Modern electronic control units and data acquisition systems continuously record parameters related to engine operation, vehicle dynamics, electrical systems, and driver inputs. However, isolated measurements provide limited information; the collected data must be processed, contextualized, and compared to support useful technical conclusions.

The analysis process generally begins with data preparation. Measurements acquired from different sensors may contain noise, missing samples, duplicated values, or inconsistent timestamps. Before any conclusions are drawn, the dataset must therefore be validated and organized. Typical preparation operations include removing invalid samples, converting values into consistent engineering units, aligning timestamps, and filtering signals when measurement noise may affect their interpretation. This stage is particularly important when data originates from multiple subsystems operating at different acquisition frequencies.

After preparation, vehicle data can be examined using statistical and time-domain analysis. Statistical methods provide a compact description of the behaviour of each signal, while time-domain analysis preserves the sequence of events occurring during a session. The combination of these approaches makes it possible to characterize normal operation, compare different tests, and identify periods that require more detailed investigation.

2.3.1 Statistical Analysis

Descriptive statistics provide an initial overview of the collected data. Measurements such as the minimum, maximum, mean, median, standard deviation, and range can be calculated for parameters including engine speed, coolant temperature, manifold pressure, throttle position, air-fuel ratio, battery voltage, and vehicle speed. These values help summarize the operating conditions observed during a session and make comparisons between different sessions more objective.

The mean represents the average operating value of a signal, while the minimum and maximum identify its observed operating limits. The standard deviation indicates how widely the measurements vary around the mean and can therefore help distinguish stable behaviour from highly variable operation. The median is also useful when a dataset contains isolated extreme values, since it is less affected by outliers than the arithmetic mean.

Statistical analysis may be performed over an entire acquisition session or over selected time intervals. Whole-session statistics provide a general summary but may hide short-duration events. Window-based analysis, in which statistics are calculated over consecutive intervals, preserves more information about changes throughout the session. For example, a stable average coolant temperature over a complete run may conceal a temporary overheating event that becomes evident when shorter time windows are examined.

In motorsport applications, the interpretation of a single parameter is often insufficient. Several signals must be considered together to understand the operating state of the vehicle. Engine speed and throttle position can indicate driver demand, while manifold pressure and air-fuel ratio provide information about engine load and combustion conditions. Coolant temperature and battery voltage can be monitored simultaneously to assess thermal and electrical stability. This combined analysis provides a more complete representation of vehicle behaviour than the isolated observation of individual measurements.

2.3.2 Session Comparison

Comparing multiple acquisition sessions is useful for evaluating changes in vehicle configuration, environmental conditions, driving behaviour, or system performance. Sessions can be compared using common indicators such as maximum engine speed, average coolant temperature, time spent within defined operating ranges, signal variability, and the occurrence of abnormal values.

For a meaningful comparison, equivalent portions of each session should be selected whenever possible. Differences in session duration, route, driver input, or operating conditions may otherwise produce misleading conclusions. Timestamp alignment and the normalization of relevant measurements can also be required when the compared datasets have different sampling frequencies or starting times.

Session comparison supports iterative vehicle development by providing objective evidence about the effect of technical changes. It may help determine whether an adjustment improved performance, reduced temperature variation, stabilized the air-fuel ratio, or introduced an undesirable operating condition. The same approach can also be used to compare the current session against a previously established reference session.

2.3.3 Anomaly Identification

Anomaly identification consists of detecting measurements or patterns that differ from the expected behaviour of the vehicle. A simple approach is based on predefined limits for each parameter. Values outside an acceptable range can be marked for later inspection, such as excessive coolant temperature, low battery voltage, an abnormal air-fuel ratio, or loss of ECU synchronization.

Statistical thresholds may also be used when fixed limits are not sufficient. Measurements that deviate significantly from the recent mean or median can indicate sudden changes, sensor errors, communication faults, or unusual operating conditions. Nevertheless, an isolated extreme value does not necessarily represent a real vehicle fault. It may result from electrical noise, an invalid CAN frame, a decoding error, or a temporary interruption in communication.

For this reason, anomaly detection should consider the duration and context of the event. A condition that persists across several consecutive samples is generally more relevant than a single invalid measurement. Correlating related signals can further improve interpretation. For example, a change in manifold pressure may be expected when throttle position changes, whereas the same variation without a corresponding driver input may justify further investigation.

The purpose of anomaly identification in the proposed platform is not to perform safety-critical diagnosis or automatically control the vehicle. Instead, it provides a method for highlighting relevant periods in the recorded data, allowing users to inspect them and make informed technical decisions.

2.3.4 Visualization of Acquired Data

Visualization complements statistical analysis by showing how vehicle parameters evolve over time. Time-series plots are particularly useful for observing transient events, identifying trends, and understanding relationships between signals. Displaying multiple synchronized signals on the same time axis enables the user to associate driver inputs with the corresponding engine and vehicle responses.

Dashboards provide an immediate representation of current operating conditions, while session plots and statistical summaries support offline analysis. These forms of visualization serve different purposes: real-time displays prioritize fast interpretation of

critical parameters, whereas offline tools allow more detailed exploration of recorded sessions.

Effective visualization should present the most relevant information without overwhelming the user. Signals should be displayed with clear names, units, scales, and timestamps. Warning limits or reference ranges may also be included to help distinguish normal values from conditions that require attention. Through the combination of real-time monitoring, session summaries, and synchronized plots, collected CAN data can be transformed into information that is more useful for vehicle evaluation and technical decision-making.

2.4 Bluetooth Integration

2.4.1 BLE Protocol

Bluetooth Low Energy (BLE) is a short-range wireless protocol designed for low-power, event-driven communication. Unlike Classic Bluetooth, BLE does not maintain a continuous data stream. Instead, it organises data exchange around the Generic Attribute Profile (GATT), where a peripheral advertises its presence and exposes data, and a central scans for and connects to it [11].

Within GATT, data is structured as services, containing one or more characteristics. Each characteristic holds a value and a set of permission flags. A `read` flag allows the central to request the current value explicitly; a `notify` flag allows the peripheral to push updates automatically once the central subscribes by writing to the characteristic's Client Characteristic Configuration Descriptor (CCCD) [12].

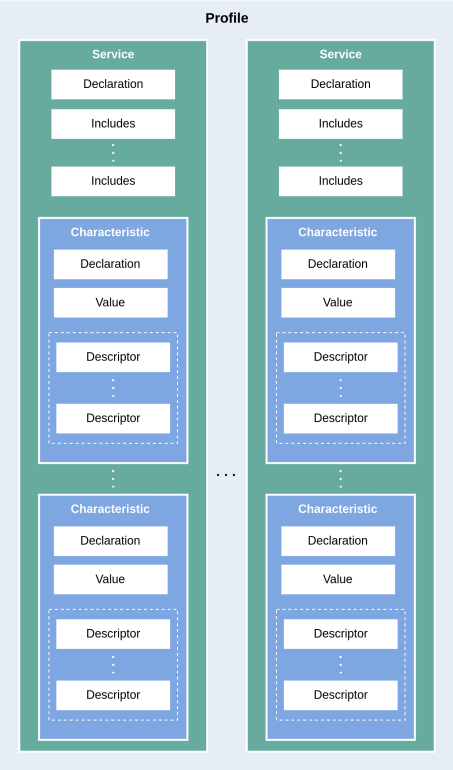


Figure 2.3: BLE GATT Hierarchical Architecture [13]

In our implementation, the *bluezero* library is used to expose the Raspberry Pi as a GATT peripheral over the Linux BlueZ stack via D-Bus. A `Peripheral` object is configured with a service and its characteristics, each assigned a Universally Unique Identifier (UUID), permission flags, and a callback. For `notify` characteristics, bluezero invokes the `notify_callback` once on subscribe and once on unsubscribe, passing the live characteristic object. Periodic transmission is then driven by a GLib timer that calls `characteristic.set_value()` at the desired interval, which BlueZ translates into a BLE notification delivered to the connected device. The server is started by calling `peripheral.publish()`, which hands execution to the GLib main loop indefinitely [14].

2.5 Project Synthesis and Differentiation

In our analysis of the current alternatives to our project, we believe our implementation provides an alternative not seen in any other similar project for several reasons:

- **Open-source:** Most alternatives are are not open-source, which make the modification or modular separation of each of it's components virtually impossible;
- **Modular Components:** Our project is separated in two different modules: Controller and Mobile App. Due to this, our project is flexible to the implementation of different alternatives as long as they are in accordance to the same communication protocol between modules;
- **Database integration:** From our research, every other alternative is a real-time monitoring. Ours gives the possibility of storing all the CAN dataframes in a database for posterior use, such as future simulations or statistical analysis;
- **Cross-plataform Accessibility:** Most options are purely limited to one plataform, be it a computer or a mobile device. Our implementation allows us to monitor the same data in different platforms at the same time, be it computer, mobile device or any other future device with a bluetooth connection;
- **Cheap and easy do aquire hardware:** Many other implementations utilize hard to source components, or may require a difficult setup.

Main close competitors to the proposed system:

- Tuner Studio Dashboard(only dashboard) [15];
- uaDash(only dashboard)[16];
- MegaLog viewer (only logger)[17].

Chapter 3

Implementation

3.1 System Architecture

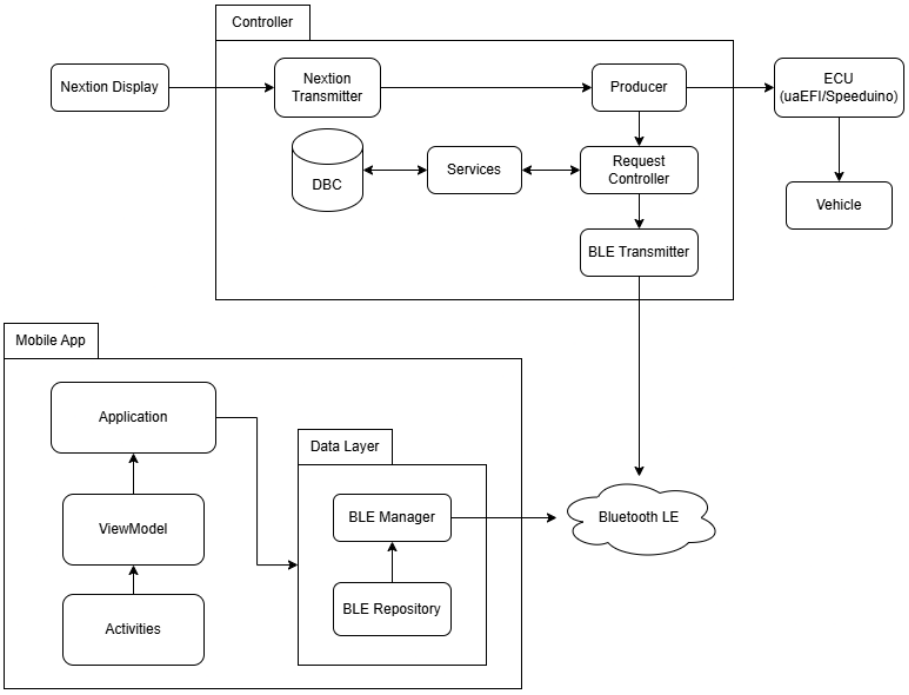


Figure 3.1: Project Architecture Diagram

Figure 3.1 illustrates the overall system architecture. The ECU communicates with the Controller via a wired CAN serial link, providing a continuous stream of raw engine frames that are decoded against the vehicle's DBC definition file. At its core sits the Controller, running on the Raspberry Pi Zero 2W, which serves as the central hub for all data acquisition and distribution. Processed data is simultaneously forwarded to a Nextion display terminal, providing immediate on-site visualisation of critical engine parameters. In parallel, the Controller operates as a Bluetooth Low Energy GATT peripheral, advertising live sensor data to any subscribed central device within range. In the current implementation, an Android mobile application acts as that central, receiving the BLE notifications and presenting the decoded signals in a real-time dashboard.

3.1.1 Controller Configuration File

The behaviour of the Controller is defined through a JSON configuration file named `config.json`. This approach separates deployment-specific settings from the application source code, allowing the same software to be adapted to different ECUs, serial interfaces, display devices, and acquisition requirements without modifying its implementation. The file is loaded during application start-up and is also read by the initialization script to determine which hardware interfaces must be available before the Controller is launched.

The `type` parameter selects the data producer. The value `rusefi_can` enables the CAN acquisition path through a GVRET-compatible serial adapter, whereas `speeduino_arduino` selects the direct Speeduino serial protocol. The `mode` parameter selects the local output interface: `nextion` starts the external Nextion display thread, while `pi_screen` starts the PyQt-based interface on a display connected directly to the Raspberry Pi. These parameters allow the acquisition source and the visualization method to be selected independently.

For the `rusefi_can` producer, `com` identifies the serial device associated with the CAN adapter and `baud_rate` defines its communication speed. The `dbc` parameter specifies the path of the DBC file used to translate raw CAN frames into physical vehicle signals. Relative DBC paths are resolved from the Controller directory during start-up. When the `speeduino_arduino` producer is selected, the equivalent serial settings are provided through `port` and `baudrate`, and `command` defines the request byte sent to the ECU before each data packet is read. Consequently, only the parameters associated with the selected producer are required during a particular deployment.

Data persistence and diagnostic output are controlled by a separate group of parameters. The `session_description` field provides a human-readable description for an acquisition session. The `state_save_interval`, expressed in seconds, determines the minimum interval between consecutive normalized vehicle-state records stored in the database. Raw CAN frames may still be processed at a higher rate, so this value controls database snapshot frequency rather than CAN bus acquisition frequency. The `log_can_activity` flag enables or disables periodic CAN activity messages, while `can_log_interval` determines how frequently the number of received, decoded, and undecoded frames is written to the application log.

The Nextion interface is configured through `nextion_port` and `nextion_baud`, which define its UART device and communication speed. The `nextion_sessions_interval` parameter determines how frequently the session list is refreshed, and `nextion_sessions_limit` defines the maximum number of sessions presented on the display. The values of `shift_point` and `redline`, both expressed in revolutions per minute, control the RPM visualization: the former determines when the shift indicator changes state, while the latter defines the upper RPM scale used by the dashboard and session graphs.

Finally, `bluetooth_enabled` determines whether the BLE server is started together with the Controller. Disabling this option allows acquisition, database storage, and local visualization to continue without exposing the GATT service. The configuration file therefore acts as the central deployment description of the platform, while default values implemented by the software are used for selected optional parameters when they are omitted.

3.2 Data Analysis Techniques in Motorsport

3.2.1 Statistical Analysis

The proposed system records a set of normalized vehicle parameters derived from CAN messages and stored periodically in the database. The currently logged parameters include:

Parameter	Description
RPM	Engine speed in revolutions per minute, primary indicator of engine operating speed.
SYNC	ECU/engine synchronization status, indicates whether the ECU has valid crankshaft and camshaft position information.
ENGINE	General engine operating status, useful for determining whether the engine is stopped, cranking, or running.
MAP	Intake manifold absolute pressure, one of the primary engine load measurements used for fuel calculations.
BARO	Ambient atmospheric pressure, used for altitude and weather compensation.
TPS	Throttle position, represents driver demand and throttle opening percentage.
IAT	Intake air temperature, affects air density and fueling calculations.
CLT	Engine coolant temperature, critical for warm-up enrichment and engine protection strategies.
AFR	Measured air-to-fuel ratio, key indicator of combustion mixture quality and engine tuning.
EGO	Fuel correction based on oxygen sensor feedback, represents closed-loop fueling adjustments applied by the ECU.
PW	Injector opening duration, directly relates to the amount of fuel delivered to the engine.
VE	Volumetric efficiency used in fuel calculations, represents how effectively the engine fills its cylinders with air.
ADV	Ignition timing advance, major factor of power output, efficiency, and knock resistance.
DWELL	Ignition coil charging time, determines the energy available for spark generation.
BAT	Electrical system voltage, important for injector operation, ignition performance, and system health monitoring.
BOOST	Target boost pressure, desired manifold pressure set by the ECU for forced-induction engines.
DUTY	Wastegate control duty cycle, indicates how aggressively the ECU is controlling boost pressure.
VSS	Vehicle speed, used for logging, speed-based strategies, and traction-related functions.
FAN	Cooling fan status, indicates whether the radiator cooling fan is active.
FP	Fuel pump status, shows whether the ECU has enabled the fuel pump output.
BOOSTCUT	Overboost protection status, indicates whether boost safety limits have been exceeded and protective intervention is active.

Table 3.1: ECU Parameters Displayed by the Data Logger

3.3 Hardware development

3.3.1 Hardware schematics

Our current hardware implementation is comprised of a Raspberry Pi Zero 2W, to which connects a nextion (via serial through the GPIO) and has a USB connection to a GVRET CAN module, which in this case, is a XIAORet, which was also developed in ISEL. [18]

The Nextion connection to the raspberry Pi is defined as such:

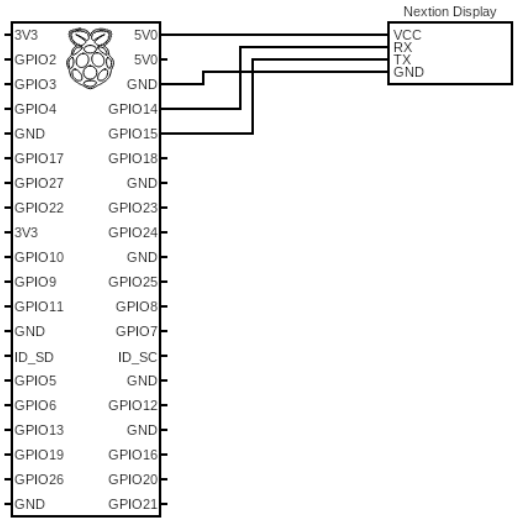


Figure 3.2: Electric diagram of the Raspberry Pi and Nextion screen

3.3.2 XIAORet comunication protocol

The XIAORet is a ESP32 microcontroller based CAN Module, that implements the SavvyCan Comunication protocol [19]. SavvyCan is widely used CAN tool for logging, receiving, sending CAN frames and much more. Its widely used in the motorsport world, Formula student and many applications where a CAN tool is needed. The SavvyCan Comunication protocol is defined as such:

- First, we need enable binary so the following hex command is sent to the XIAORET:0xE7

- Secondly, we loop through the received CAN frames, parsing each message. For each frame, the system extracts the timestamp, CAN identifier, data length code (DLC), and payload bytes from the serial stream. The extracted identifier is then processed using a bitmask to ensure compatibility with both standard and extended CAN formats. Subsequently, the frame is matched against the DBC definition, allowing the raw payload to be decoded into meaningful vehicle signals. When a valid mapping is found, the decoded values are integrated into the internal ECU state representation; otherwise, the frame is preserved in its raw form to maintain traceability and support debugging.

- Although the presented sollution supports 29bit CAN Extended frames, only 11bit CAN Standard frames were tested.

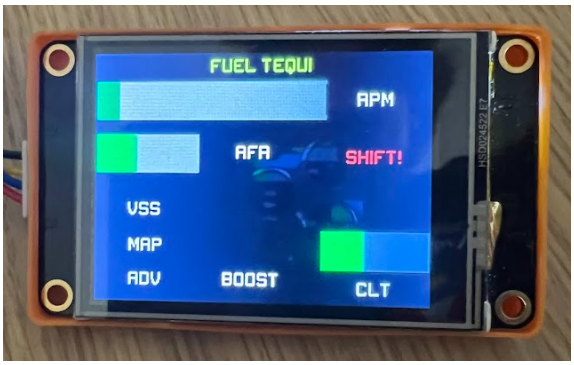


Figure 3.3: Nextion screen 1

Figure 3.4: Nextion screen 2

3.4 Software Implementation

3.4.1 Python Controller

The Python Controller is responsible for several different tasks such as:

- **CAN Connection:** Connection between the ECU module and the Python application using a serial USB link;
- **Database Manager:** Control the database interactions, storing or retrieving the necessary data;
- **UI Display:** Present the information required using a Nextion device, be it real-time from cache or through database requests;
- **Bluetooth Connection and Data Relay:** Connection between the Python application and a connected Bluetooth device (using a mobile app in our implementation).

For that end, the application itself has several different inner modules that we will now breakdown and explain, starting with the domain and repository layers.

The **Domain** and **Repository** layers are working as a pair since the domain defines the way each data structure is implemented and the repository defines how it gets stored and retrieved. Neither are responsible for the restriction logic - that will be the responsibility of *Services*.

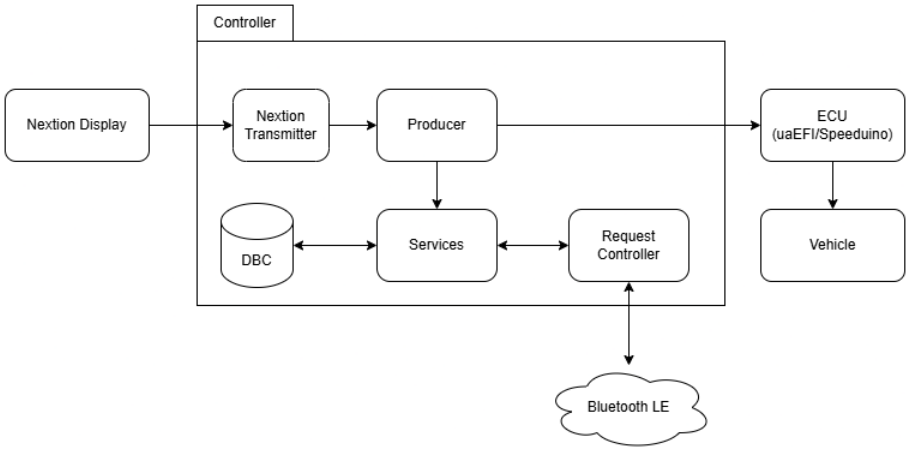


Figure 3.5: Controller Structure Diagram

The domain consists of four different types of classes - `Session`, `CANFrame`, `Signal` and `VehicularState` - each used simply to keep information that is either being retrieved or stored in the database.

A session represents one continuous recording period with a start timestamp, an optional end timestamp that begins as None, and a human-readable description. A `CANFrame` captures a single raw message off the bus: which session it belongs to, when it arrived, the CAN arbitration ID, the data length code, and the raw data bytes as a string. A Signal is the decoded output of a frame, associating a signal name and its parsed numeric value and unit back to the frame it came from. A `VehicularState` records the current dataframe state of the vehicle for easy access to the complete information of the vehicle in a specific timestamp. Each domain class has it's respective repository class that will handle the requests to the database.

The repository layer sits directly on top of SQLite database and mirrors that domain structure previously stated. Each repository module opens its own connection the database at import time using a module-level `.connect()` call, then exposes a set of plain functions rather than a class. Each module owns its own conn, which means closing the database requires calling `.close()` on four separate objects through `Services .close_database()`. This also means that any cross-table operation that needs to be atomic — such as inserting a frame and its signals together — has no transaction boundary protecting it. That is an architectural limitation worth addressing before the project handles high-frequency acquisition in production.

The services module acts as the single entry point through which the rest of the application interacts with the database. Rather than letting the acquisition pipeline, the UI, or any other consumer reach into a repository directly, everything that needs to read or write persistent data goes through here. In practice every function in the module is a one-liner that receives arguments, forwards them to the appropriate repository function, and returns whatever the repository gives back. The value of this pattern is not in the logic it adds — there is none — but in the boundary it creates: if a repository is ever renamed, replaced, or refactored, only this file needs to change.

We have a class `signal_cache` that will be the singleton class that will record and transmit the lastest dataframe obtain from the ECU and relay it to both the UI Display and the Bluetooth transmission for a faster access to the real-time information.

The BLE transmitter, our bluetooth layer, turns the device into a BLE GATT that an android device is able to discover, connect to and subscribe to continuos sensor data acquired by the ECU. For it's proper operation, we create an instance of *BLElmServer*, which will build the GATT tree by creating a `Peripheral` object bound by the device's Bluetooth

adapter MAC address and advertising under the local name "*Vehicular_Monitor*" and will register one primary service and three characteristics under that service. The first characteristic will be configured with both `read` and `notify` flags in order for the mobile client to either poll the data explicitly or to be notified of any updates to the signal cache. This way we can attach two different callback: `read_callback` and `notify_callback`. The second characteristic will be configured with `write` flag in order to obtain session requests from the mobile client and has a `_session_request_cb` as a `write_callback`. This characteristic exists to be able to accurately send requested information through the third characteristic. The third characteristic is similar to the first one with different callbacks: `_session_response_read_cb` and `_session_response_notify_cb`

The notify mechanism is the most important part to understand correctly. In *Bluezero*, `notify-callback` is not a function that gets called each time a notification needs to be sent — it is a status callback that fires once when a client subscribes and once when it unsubscribes. When it fires with `notifying=True`, the callback receives a reference to the live characteristic object and immediately schedules a repeating GLib timer set to fire every 200 milliseconds. Each time the timer fires, `_send_notify` calls `characteristic.set_value()` with the latest encoded cache string, which emits a D-Bus `PropertiesChanged` signal that BlueZ translates into an actual BLE notification packet sent to the Android device. The timer returns True on every successful call to keep itself alive, and only returns False — which removes it — if the characteristic reference has been cleared. When the client unsubscribes and notify_cb fires with notifying=False, the stored characteristic reference is cleared and the GLib timer is cancelled with GLib.source_remove, stopping all further transmissions until a new subscription arrives.

As mentioned before, the BLE transmitter utilizes two characteristics in charge of obtaining session requests and returning session responses towards the remote device. These characteristics allow the use of several different commands: the session list (format: `"GET_SESSIONS"`), session information by identifier (format: `"GET_SESSION:_ID_"`), session creation (format: `"CREATE_SESSION"`), and session ender (format: `"END_SESSION"`). These commands either return a list of values obtained from the database, or a boolean in case of error, and are available to any application that is connected to the available service provided by the python controller and is connected through BLE.

Calling start() triggers ble.publish(), which begins advertising the peripheral and hands control to the GLib main loop. This call never returns under normal operation — everything that needs to happen alongside the BLE server, such as the mock data generator or the console status printer, must be running in daemon threads before start() is called. The process must also be launched with *sudo ./venv/bin/python3* rather than bare sudo python3, because BlueZ requires root privileges and the system Python does not have bluezero installed.

3.4.2 Python Producer

The Python producer application constitutes the core of the proposed CAN platform controller, being responsible for data acquisition, processing, storage, and visualization. The system is implemented in Python and follows a modular, multi-threaded architecture designed to ensure high performance, scalability, and maintainability. Both the controller are integrated and running in the same processor as a way to not only obtain all necessary information through the producer and implement all required functionality through the controller.

From an architectural perspective, the application adopts a distributed processing model based on multiple worker threads. This approach allows different responsibili-

ties such as data acquisition, decoding, persistence, and visualization to be executed concurrently. As a result, the system minimizes bottlenecks and ensures real-time responsiveness when handling high-frequency CAN data streams.

The application entry point is defined in *main.py*, which orchestrates the system initialization workflow. At startup, the application reads a configuration file (*config.json*) that defines both the data source (*producer type*) and the output interface (*mode*). This configuration-driven design enables flexibility, allowing the same software to operate with different ECUs and visualization methods without requiring code modifications.

Following configuration loading, the system initializes the SQLite database (*ecu_data.db*) and ensures that all required tables are created. These include tables for raw CAN frames, acquisition sessions, and normalized vehicle state snapshots. The database layer is abstracted through a repository pattern, which decouples data persistence from the rest of the application logic and improves maintainability.

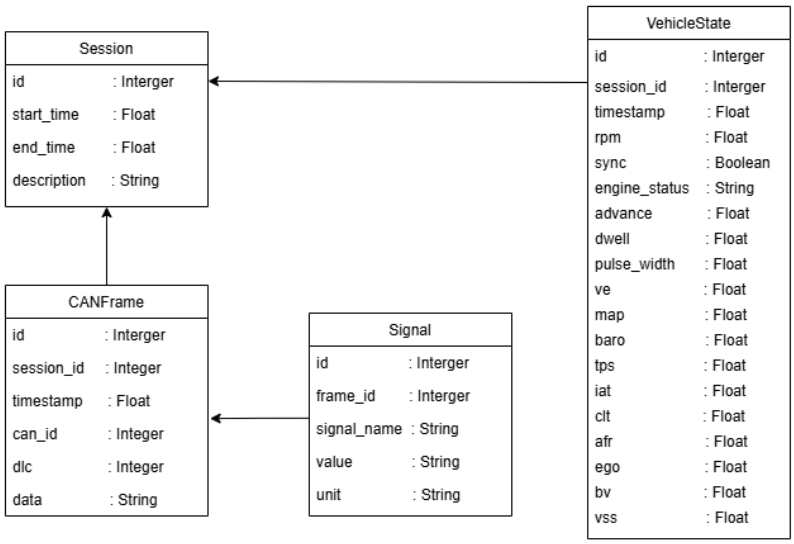


Figure 3.6: Database Diagram

The data acquisition process is handled by a dedicated producer thread, which can operate in different modes depending on the selected ECU interface. In the *rusefi_can* mode, the system reads CAN frames via a SavvyCan-compatible serial adapter. These frames are decoded using a DBC file, allowing raw binary messages to be translated into meaningful automotive signals. Alternatively, in the *speeduino_arduino* mode, the application directly parses structured serial data packets via USB, from the Speeduino ECU.

Decoded signals are then mapped into a unified internal representation of the vehicle state. This state is maintained in a shared memory structure called *signal_cache*, which always stores the most recent values received from the ECU. This design avoids repeated database queries and enables fast access for visualization components.

In parallel with the acquisition process, additional threads handle data persistence and output. The system periodically stores snapshots of the vehicle state in the database, enabling offline analysis while retaining the raw CAN traffic required for traceability.

3.4.2.1 CAN Data Persistence and Session Recording

Database recording is organized around acquisition sessions. The Controller continues to acquire and decode CAN traffic immediately after start-up, but persistent recording only begins when a session has been created. In the present Nextion workflow, the display

starts a session by transmitting a NEW_SESSION request. If another session is already active, its end timestamp is recorded before a new row is inserted into the sessions table. Each subsequent database record is associated with the identifier of the active session. Before this request is received, decoded data remains available to the live display and Bluetooth interface through the signal cache, but it is not written to the session tables.

Two complementary forms of data are stored during a session. First, every valid GVRET CAN frame is inserted into the can_frames table, including frames whose identifiers are absent from the DBC file or whose payload cannot be mapped to the normalized vehicle state. Each row contains the session identifier, reception timestamp, CAN identifier, data length code, and hexadecimal payload. This provides a complete record of the received bus traffic and preserves undecoded frames for subsequent inspection or reprocessing. The current implementation performs and commits each insertion individually rather than grouping several frames into a transaction.

Second, decoded frames update an in-memory aggregate containing the latest known value of every supported vehicle parameter. Since the RusEFI base messages distribute related signals across several CAN identifiers, an individual decoded frame generally modifies only part of this aggregate. For example, one message updates engine speed, ignition advance, and vehicle speed, while another updates manifold pressure and temperatures. Values obtained from earlier messages are retained until they are replaced, allowing the aggregate to represent the most recent complete vehicle state.

A copy of this aggregate is periodically inserted into the vehicle_state table. The minimum interval between consecutive snapshots is defined by the state_save_interval configuration parameter, expressed in seconds. With a value of 1.0, the system records approximately one vehicle-state row per second; values of 0.5 and 0.25 allow maximum rates of approximately 2 and 4 Hz, respectively. The interval is evaluated when a decoded frame arrives, rather than by an independent timer. Consequently, a snapshot is written on the first decoded frame received after the configured interval has elapsed. No state is stored during periods in which no decodable traffic is received, and the actual spacing may therefore be slightly greater than the configured value.

The snapshot interval is reset whenever the active session changes, causing the first decoded frame of a new session to be recorded immediately. Each vehicle_state row contains the session identifier, insertion timestamp, and the complete normalized set of engine, load, temperature, fuelling, ignition, electrical, boost, speed, and status parameters. Its timestamp is generated by the Raspberry Pi at insertion time. By contrast, raw CAN frame rows use the frame reception time captured by the acquisition process. The adapter-provided GVRET timestamp is used during parsing but is not presently persisted as a separate database field.

This separation provides different resolutions for different purposes. The can_frames table follows the incoming bus rate and maintains the detailed source record, whereas vehicle_state contains a lower-frequency time series suitable for session graphs and general analysis. For example, a ten-minute session with a snapshot interval of one second produces approximately 600 vehicle-state rows, while the number of raw frame rows depends directly on CAN traffic and may be several orders of magnitude higher. Session graph requests query vehicle_state, filter rows by session identifier, and order them by timestamp before the selected parameters are plotted.

The database schema also contains a signals table intended to associate individually decoded values with their source CAN frames. This table is supported by the repository layer but is not populated by the current GVRET acquisition path; normalized values are instead retained through the periodic vehicle_state snapshots. Furthermore, equivalent persistence has not yet been integrated into the speeduino_arduino reader, which currently updates the live signal cache without creating raw-frame or vehicle-state records.

These constraints define the present recording scope and identify areas for future development, particularly transaction batching, explicit session termination during shutdown, and a common persistence path for both supported producers.

For visualization, the application supports two independent output modes. In the *Nextion* mode, a dedicated thread continuously reads the latest values from the *signal_cache* and transmits them over a serial interface to a Nextion display, providing a lightweight, embedded graphical interface. In the *pi_screen* mode, a PyQt-based dashboard runs locally on the Raspberry Pi, periodically refreshing the displayed values using a timer mechanism. The separation between data acquisition and visualization ensures that user interface performance does not interfere with real-time data processing.

3.4.2.2 Nextion Session Graph Generation

In addition to displaying live values, the Nextion interface can plot signals stored during a previous acquisition session. A graph request issued by the display identifies the selected session and the signals to be represented. The Controller resolves the displayed session index to its database identifier, retrieves the corresponding vehicle-state records, and converts the resulting series into native Nextion drawing commands. The graph is therefore rendered by the display itself using line, fill, and text commands, without transferring a bitmap or relying on the Nextion waveform component.

The plotting area begins at coordinate $(35, 45)$ and has a width of 410 pixels and a height of 170 pixels. Samples are distributed uniformly along the horizontal axis, from $x = 35$ to $x = 445$, according to their order in the retrieved series. For a series containing N samples, the horizontal coordinate of sample i is given by

$$x_i = 35 + \left\lfloor \frac{i}{N-1} \times 410 \right\rfloor . \tag{3.1}$$

This representation assumes that stored vehicle-state samples are separated by approximately regular intervals. When a session contains more than 410 records, the series is reduced to 410 uniformly selected samples. Both endpoints are retained, ensuring that the complete session is represented while limiting the number of serial commands and avoiding multiple samples being assigned to the same horizontal pixel.

Each signal is normalized independently on the vertical axis using a predefined physical range. If v_i is the measured value, and $v_{\min}$ and $v_{\max}$ are the limits assigned to that signal, its vertical coordinate is calculated as

$$y_i = 45 + 170 - \left\lfloor \frac{v_i - v_{\min}}{v_{\max} - v_{\min}} \times 170 \right\rfloor . \tag{3.2}$$

Values outside the specified range are clamped to the upper or lower graph boundary. The inversion in this expression is required because the Nextion coordinate system places the origin at the top-left corner of the screen. Consequently, a signal's minimum value is plotted at $y = 215$, its midpoint at approximately $y = 130$, and its maximum at $y = 45$.

The ranges used by the implementation are listed in Table 3.2. The RPM upper limit is obtained from the configurable `redline` value; the remaining limits are fixed according to the expected operating range of each parameter.

Signal	Minimum	Maximum	Unit
Engine speed (RPM)	0	`redline`	rpm
Air–fuel ratio (AFR)	10	20	–
Coolant temperature (CLT)	0	120	°C
Intake air temperature (IAT)	0	80	°C
Throttle position (TPS)	0	100	%
Manifold pressure (MAP)	0	250	kPa
Barometric pressure (BARO)	80	120	kPa
Ignition advance (ADV)	-20	60	degree
Injector pulse width (PW)	0	20	ms
Battery voltage (BAT)	0	16	V
Boost target	0	250	kPa
Boost duty	0	100	%
Vehicle speed (VSS)	0	250	km/h

Table 3.2: Vertical scales used for Nextion session graphs

After coordinate conversion, consecutive points belonging to the same signal are joined by line commands. Up to six signals can be distinguished using the predefined red, blue, green, orange, cyan, and magenta sequence; colours are reused if additional signals are requested. Since every signal uses its own vertical scale, overlapping curves indicate a similar relative position within their respective operating ranges rather than equality of their physical values. This normalization allows parameters with different units and magnitudes, such as RPM, AFR, and coolant temperature, to be inspected together within the limited display area.

To support deployment as an embedded system, the application includes an initialization script (*init.sh*) designed to be executed as a systemd service (or first time startup). This script prepares the runtime environment, waits for required hardware interfaces (such as serial devices), and guarantees automatic startup and recovery in case of failure. This makes the system robust and suitable for unattended operation in automotive environments.

Overall, the combination of a multi-threaded architecture, modular design, configuration-driven behavior, and efficient data handling mechanisms allows the Raspberry Pi application to meet the real-time requirements of vehicular data acquisition systems while maintaining flexibility and extensibility for future developments.

3.4.3 Android application

The Android application is utilized as an exterior module to the Python Controller that will receive data via BLE from the controller and display it in an intuitive manner. This module could be replaced for any other application as long as it follows the communication protocol defined by the controller and be connected to the controller via Bluetooth.

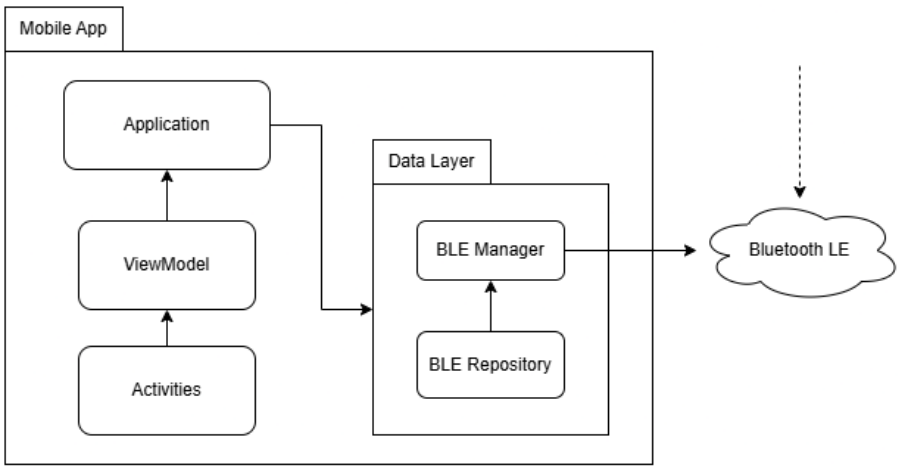


Figure 3.7: Mobile Application Structure Diagram

The structure of the Android application is as follows:

- **Application:** Root of the entire dependency structure and implements the `Dependency Container` interface that declares three different singletons - `Ble Manager`, `Ble Repository` and `BleViewModelFactory` - available to the rest of the application
- **Activity:** All activities extend `BaseActivity` and are the boundary between the application and it's UI. `BaseActivity` standardizes lifecycle logging across every screen and exposes a generic `navigate<T>()` helper used to move between activities, as well as a backPressed()/backAware() hook for consistent back-navigation handling. The `MenuActivity` is the primary activity. It manages the application permissions (`BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` and `ACCESS_FINE_LOCATION`) and starts the scanning for connected Bluetooth devices, working as a link to distribute said Bluetooth device's information to the rest of the possible activities. Selecting a scanned device opens `DeviceMenuActivity`, a per-device menu (Live Data, Sessions Information, Overall Statistics, Device Information) that in turn opens `DataDisplayActivity` to show the live data stream for that device. `AboutActivity` is reachable from `MenuActivity` and shows application information.
- **ViewModel:** The core of the application's state management through `BleViewModel`. It holds the current state `_uiState` of the current application and exposes it in a immutable *StateFlow* to the current observable screen. When `.start()` is called, it launches a coroutine that collects cvs strings from `BleRepository.canData` and when `.connect()` is called it passes the selected device to the repository to initiate GATT connection. Each incoming csv line is parsed by `parseCanData()` into a `MainUiState` carrying the individual channels reported by the controller (RPM, coolant temperature, AFR, TPS, MAP, battery voltage, dwell, ignition timing, VSS, IAT, EGO correction, VE, and sync status), with the raw string preserved alongside the parsed fields. For development and demonstration without a physical controller connected, `BleViewModel` also exposes `.startMockData()`, which generates

a continuous stream of plausible randomized values into the same uiState so the UI can be exercised standalone. `BleViewModelFactory` allows the construction of a `BleViewModel` with the `BleRepository` constructor.

- **Model:** The `BleManager` owns every interaction with the Android Bluetooth stack, scanning for devices, loading already-paired (bonded) devices, and connecting over GATT using a fixed service/characteristic UUID pair that forms part of the defined communication protocol with the controller. `BleRepository` wraps `BleManager` as a stable boundary. It re-exposes `devicesFlow` and `dataFlow` as `scannedDevices` and `canData` respectively, and delegates `startScan` and `connectToDevice` directly. Its presence means the `ViewModel` layer imports nothing from `BleManager` directly, and the BLE implementation could be replaced or mocked entirely without touching a single `ViewModel`.

Chapter 4

Results and validation

This chapter presents the validation of the platform across two test environments: Bench and Vehicle, followed by a dedicated verification of the mobile application and a synthesis of the overall results.

For this validation, the experimental hardware utilized is the following:

- uaEFI [8]
- Raspberry Pi Zero 2W (with headers) [10]
- Motorola G35 5G [20]

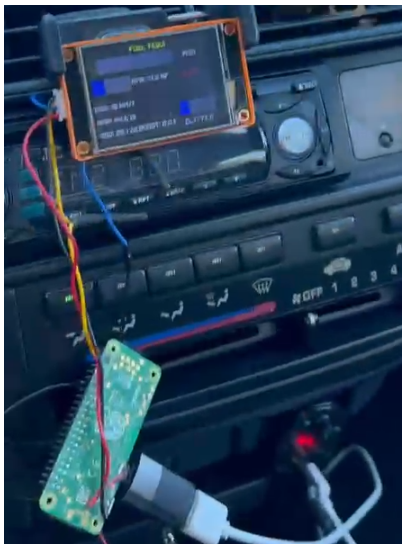


Figure 4.1: Hardware utilized

4.1 Bench Validation

Bench validation was performed with the Raspberry Pi connected via USB serial to a Speeduino ECU running on a static test bench, before any vehicle integration. Each software module was verified in isolation and as part of the integrated pipeline. No bench validation was performed with a RusEFI ECU due to testing contraints.

The CAN acquisition pipeline was confirmed to correctly decode all signals defined in the DBC file, including RPM, coolant temperature, MAP, TPS, AFR, ignition advance, dwell, and battery voltage. Decoded values were cross-checked against the reference values programmed into the ECU, with no decoding errors observed over a 30-minute continuous run.

The `SignalCache` singleton was verified for thread-safety under concurrent producer and consumer access. The alias synchronisation mechanism was also validated, confirming that updates to canonical keys such as `clt` and `advance` were correctly reflected in their aliases `temp` and `timing`.

Database persistence was validated through a complete session lifecycle: creation, continuous frame and signal insertion over 10 minutes, and termination. All records were confirmed intact, and session retrieval queries returned results in the correct order.

The BLE stack was validated using `test_ble_transmitter.py`, which populated the signal cache with mock sensor data at 5 Hz and transmitted it to the Android device. Notifications arrived at the expected 200 ms interval with all 13 payload fields correctly received across a 20-minute streaming session.

4.2 Vehicle Validation

Vehicle validation was performed on a road vehicle equipped with a RusEFI ECU. The Raspberry Pi was mounted in the cabin with a CAN connection to the ECU serial output.

Signal values from the platform were compared against TunerStudio running simultaneously on a laptop connected to the same ECU. RPM readings matched within ± 5 RPM at stable engine speeds. Coolant temperature correctly tracked the warm-up curve from cold start to operating temperature. AFR and EGO correction values were consistent with the closed-loop behaviour visible in TunerStudio.

A 5-minute driving session produced approximately 1000 CAN frames stored without write errors or data loss. Session timestamps correctly delimited the recording window. No crashes, BLE disconnections, or process restarts were observed throughout. CPU utilisation remained below 60% across all concurrent tasks.

4.3 Mobile Application Verification

The Android application was verified in three modes: mock device, bench BLE, and vehicle BLE.

Device discovery correctly distinguished between paired-only and actively advertising peripherals, with paired-only device cards rendered as non-interactable. All 13 sensor fields updated correctly at 10 Hz in the live data screen. Displayed values were validated against the bench test harness console output and against TunerStudio during vehicle testing, with no parsing errors or field misalignments observed.

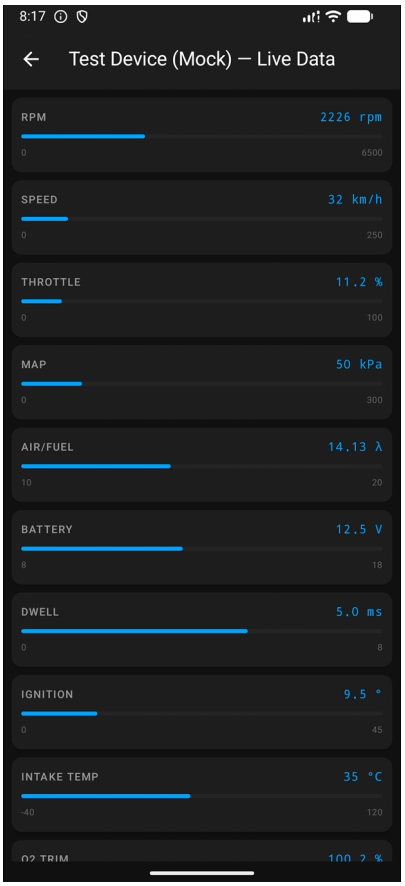
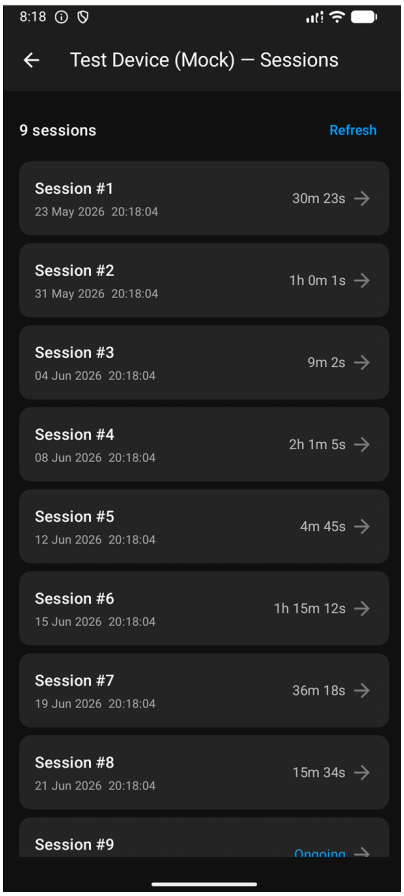
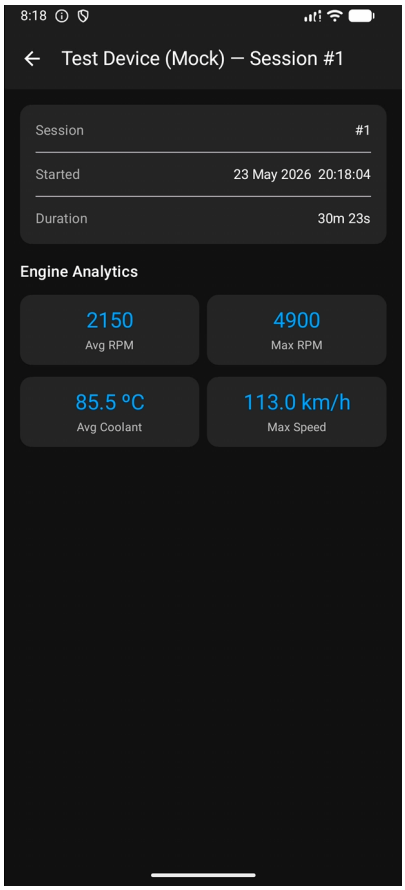


Figure 4.2: Live data display

Session acquisition and start time recording were verified to be accurate, with each driving session correctly created and timestamped upon data collection. The session-specific statistics presented by the application were confirmed to match the corresponding values stored in the database, indicating that data acquisition, storage, and retrieval operated correctly throughout the testing process.



(a) Sessions display



(b) Session information display

Figure 4.3: Session displays

The overall statistics screen was also validated by comparing its aggregated values with the complete set of session records stored in the database. The application consistently displayed accurate cumulative statistics, confirming the correctness of the aggregation logic and the integrity of the stored data.

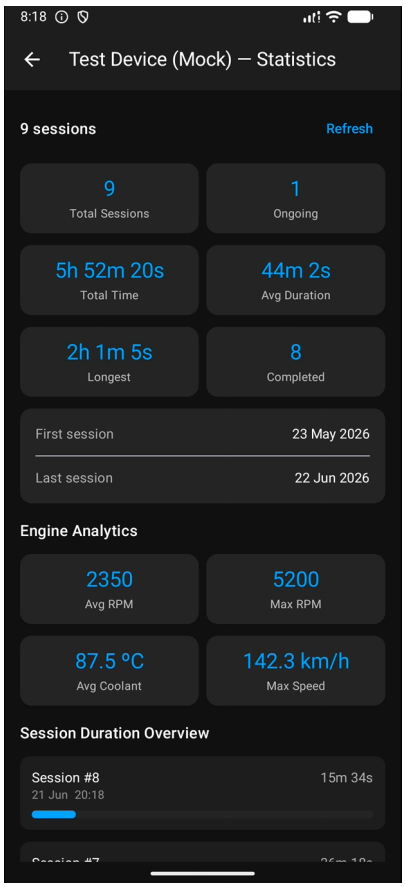


Figure 4.4: Overall statistics display

4.4 Raspberry Pi/Nextion Application verification

4.5 Results Summary

Validation confirmed that the platform meets its primary objectives. CAN acquisition and DBC-based decoding operated correctly under real engine conditions. Continuous datalogging handled high-frequency writes without data loss. BLE transmission reliably delivered a 13-signal payload at 5 Hz, and the mobile application correctly displayed all fields.

The main limitation identified is the absence of automatic BLE reconnection logic on both the controller and the Android client, currently requiring manual intervention after an unexpected disconnection. This is the primary target for improvement in future iterations of the platform.

Chapter 5

Conclusions

This work presented a CAN-based data acquisition system implemented on a Raspberry Pi, combining real-time frame acquisition, DBC-based decoding, and structured data storage. The system demonstrates that it is possible to build a functional telemetry platform using low-cost hardware, while maintaining sufficient performance for typical automotive monitoring tasks.

However, several limitations are evident. The current architecture does not enforce strict real-time guarantees, as Python threading and the underlying operating system scheduling can introduce non-deterministic delays. This limits the system's suitability for safety-critical applications.

The abstraction of CAN data into a normalized vehicle state simplifies usage but also reduces flexibility, since only a subset of signals is retained. Any changes to the monitored parameters require modifications in multiple components, including decoding logic and database schema, which impacts maintainability. Furthermore, the system does not currently distinguish between standard and extended CAN frames at a semantic level, which may be relevant in more complex network configurations.

From a data analysis perspective, the platform focuses on acquisition and storage, with limited built-in analytical capabilities. While it enables offline analysis, it does not yet exploit advanced techniques such as anomaly detection, correlation analysis, or predictive models. As a result, much of the potential value of the collected data remains unexplored.

Despite these constraints, the system provides a solid foundation for telemetry in non-critical environments such as entusiast motorsports and Formula Student. It enables consistent data collection, supports debugging and performance evaluation, and improves visibility over ECU behaviour. Future work should focus on improving data throughput, introducing more robust real-time handling, and extending the analytical capabilities of the platform. Future work should focus on improving data throughput, introducing more robust real-time handling and error detection, and extending the analytical capabilities of the platform. In particular, the development of dedicated statistical analysis tools would allow engineers to extract more meaningful insights from the collected telemetry data and support data-driven decision making. Support for Extended CAN (CAN 2.0B) frames should also be incorporated to improve compatibility with a wider range of automotive and motorsport electronic systems. Additionally, integrating positioning and motion sensors, such as GPS and Inertial Measurement Units (IMUs), would significantly enhance the telemetry platform by enabling vehicle trajectory tracking, dynamic behaviour analysis, and more comprehensive performance evaluation. These improvements would contribute to transforming the system from a data acquisition solution into a more complete vehicle monitoring and analysis platform.

Bibliography

[1] W. B. Ribbens, *Understanding Automotive Electronics, 8th Edition*. Butterworth-Heinemann, 2017.

[2] International Organization for Standardization, “Road vehicles – controller area network (can) – part 1: Data link layer and physical signalling,” International Organization for Standardization, Geneva, Switzerland, International Standard ISO 11898-1:2015, 2015.

[3] Bijal Parikh, “What is can protocol? complete guide to controller area network,” Engineers Garage, 2024. Accessed: Jul. 8, 2026. [Online]. Available: https://www.engineersgarage.com/can-protocol-understanding-the-controller-area-network-protocol/

[4] Harry, “Can standard data frame format: A simplified guide with real world examples,” 2025. Accessed: Jul. 8, 2026. [Online]. Available: https://medium.com/@tempotales1225/can-standard-data-frame-format-a-simplified-guide-with-real-world-examples-dc1a56ecaba2

[5] Vector Informatik GmbH, *Candb++ – network database editor*, Vector Informatik GmbH, 1995.

[6] O. Pfeiffer, A. Ayre, and C. Keydel, *Embedded Networking with CAN and CANopen*. Copperhill Media, 2008.

[7] Robert Bosch GmbH, “Can specification version 2.0,” Robert Bosch GmbH, Stuttgart, Germany, 1991.

[8] rusEFI Project. “Rusefi – the most advanced open source ecu,” Accessed: Jul. 8, 2026. [Online]. Available: https://rusefi.com/

[9] rusEFI Project. “Rusefi firmware source code,” Accessed: Jul. 8, 2026. [Online]. Available: https://github.com/rusefi/rusefi

[10] Raspberry Pi Ltd, *Raspberry pi zero 2 w*, 2024. Accessed: Jul. 8, 2026. [Online]. Available: https://pip-assets.raspberrypi.com/categories/584-raspberry-pi-zero-2-w/documents/RP-008359-DS-1-raspberry-pi-zero-2-w-product-brief.pdf

[11] Bluetooth Special Interest Group, *Bluetooth® low energy primer*, 2023. Accessed: Jul. 8, 2026. [Online]. Available: https://www.bluetooth.com/

[12] Bluetooth Special Interest Group, “Bluetooth core specification version 5.4,” Bluetooth Special Interest Group, 2023. Accessed: Jul. 8, 2026. [Online]. Available: https://www.bluetooth.com/specifications/specs/core-specification-5-4/

[13] Espressif Systems (Shanghai) Co., Ltd., “Bluetooth low energy - introduction,” Espressif Systems (Shanghai) Co., Ltd., 2016-2026. Accessed: Jul. 8, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/ble/get-started/ble-introduction.html

[14] Bluezero Project. “Bluezero: Python library for bluetooth low energy (ble) on linux,” Accessed: Jul. 8, 2026. [Online]. Available: https://github.com/ukBaz/python-bluezero

[15] EFI Analytics. “Tunerstudio ms dashboard,” Accessed: Jul. 8, 2026. [Online]. Available: https://www.tunerstudio.com/index.php/products

[16] rusEFI Project. “Uadash,” Accessed: Jul. 8, 2026. [Online]. Available: https://github.com/rusefi/uaDASH/

[17] EFI Analytics. “Megalogviewer,” Accessed: Jul. 8, 2026. [Online]. Available: https://www.efianalytics.com/MegaLogViewer/

[18] Itead Studio. “Iteadlib arduino nextion,” Accessed: Jul. 8, 2026. [Online]. Available: https://github.com/itead/ITEADLIB_Arduino_Nextion

[19] Collin Kidder. “Savvycan,” Accessed: Jul. 8, 2026. [Online]. Available: https://www.savvycan.com/

[20] Motorola Mobility, LLC. “Specifications - moto g35 5g,” Accessed: Jul. 8, 2026. [Online]. Available: https://en-emea.support.motorola.com/app/answers/detail/a_id/184470/~/specifications---moto-g35-5g